MCP GOVERNANCE GATEWAY

Aegis Gateway 2.0

Let AI agents reach real systems with confidence. Aegis gives every MCP call one governed route: typed tools, independent approvals, destructive-action receipts, egress control, output declassification, and verifiable audit, applied before anything touches a backend.

Apache-2.0 | Independent project (io.github.vaquarkhan) | Own host or Apache Knox | Fail-closed by default

IN ONE SLIDE

Safe agent autonomy, without rewriting your systems.

Aegis is a configuration-driven governance plane that sits between AI agents and the tools, data and services they call over MCP. It turns raw agent power into controlled, approved, audited actions, and it drops in front of an existing MCP server or an Apache project with no change to the governed code.

Control
Identity, policy, scope and approval on every call, fail-closed.

Confidence
Hash-chained audit and provenance make every action reviewable.

Coverage
13 of 20 OWASP MCP and LLM risks closed on the gateway path.

Proof
Real exploits for three MCP CVE classes denied in a live run.

THE OPPORTUNITY

MCP unlocks agent value. Governance lets you scale it.

The Model Context Protocol has made agents genuinely useful: one standard way to call tools, read files, query databases and reach services. Teams that add a governance layer get to say yes to agent automation across more systems, faster, because each new tool arrives already controlled.

Say yes, safely
Onboard a new MCP server or REST tool behind one reviewed configuration, with controls already applied.

Move fast
Typed contracts and reusable profiles mean a new integration is configuration, not a security project.

Prove it
Every decision is recorded, so adoption comes with an audit trail from day one.

Aegis meets the moment: as MCP adoption grows, Aegis is the layer that makes broad, confident rollout practical.

THE PRODUCT

A configuration-driven decision plane for MCP calls.

Aegis sits between the agent and the backend. Every tool, resource, prompt and completion call passes a fail-closed engine that applies identity, policy, scope, approval, egress, screening, output control and audit before reaching a real system.

- One configuration file drives it; the governed project is untouched.
- Typed capability contracts keep tools precise and predictable.
- Independent approval and dry-run receipts for consequential actions.
- Hash-chained audit with intent recorded before execution.

A clean fit in your stack

Aegis governs the action and data-access boundary, the point inside a single MCP JSON-RPC body where tool, method, arguments and schema live. It complements your WAF, API gateway, identity provider and SIEM rather than competing with them.

Independent project under Apache-2.0. It runs on its own or inside an Apache Knox topology.

WHY TEAMS CHOOSE IT

Eight strengths, one gateway.

Fail-closed
Nothing runs unless a control allows it.

Human in the loop
Independent approval for destructive actions.

No token passthrough
Backends get a scoped, gateway-issued credential.

Egress control
Pinned DNS and an allow list stop data exfiltration.

Output safety
Redaction and handle-only keep raw text from the model.

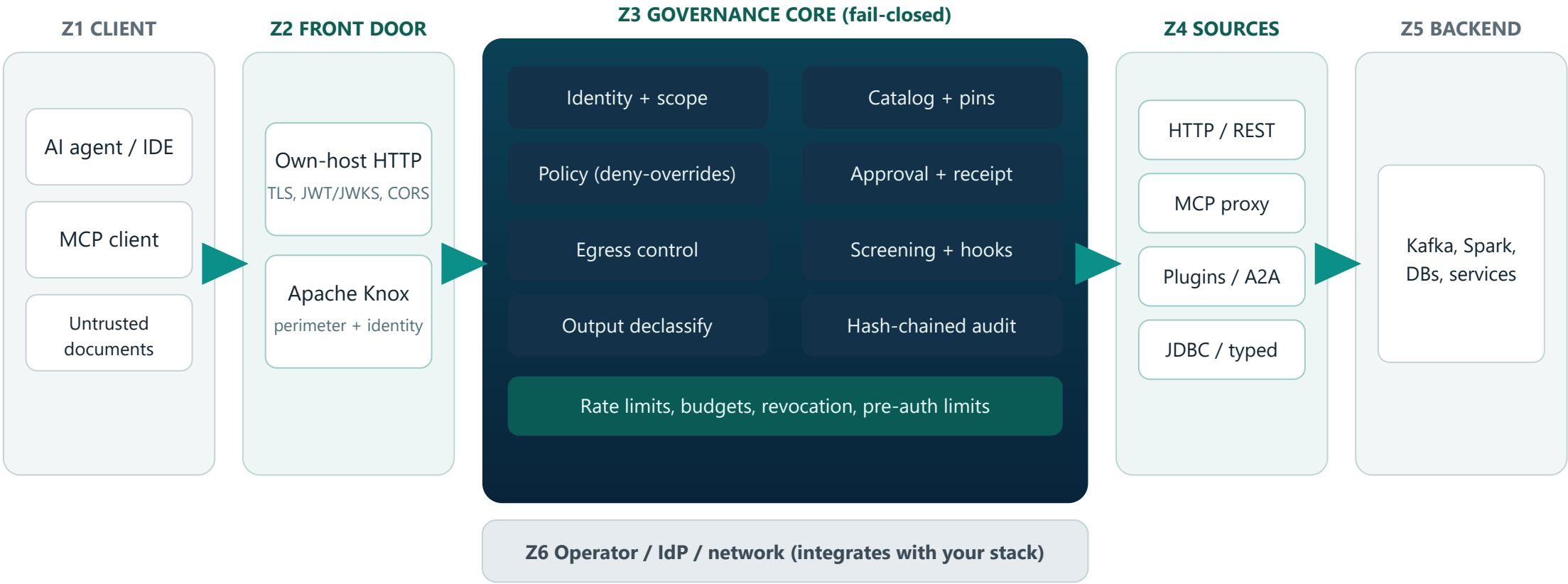
Tamper-evident audit
Hash-chained journal, signed checkpoints, SIEM sink.

Runs anywhere
One image on EKS, ECS, EC2 or your own host.

Drop-in adoption
Front any MCP server or Apache project by config.

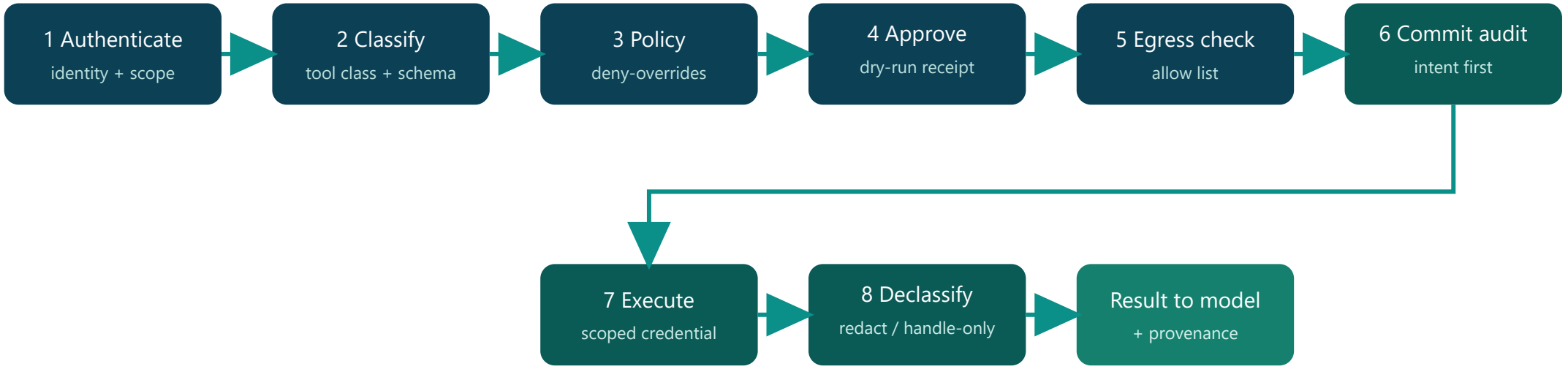
ARCHITECTURE

Six trust zones, one fail-closed engine.



REQUEST LIFECYCLE

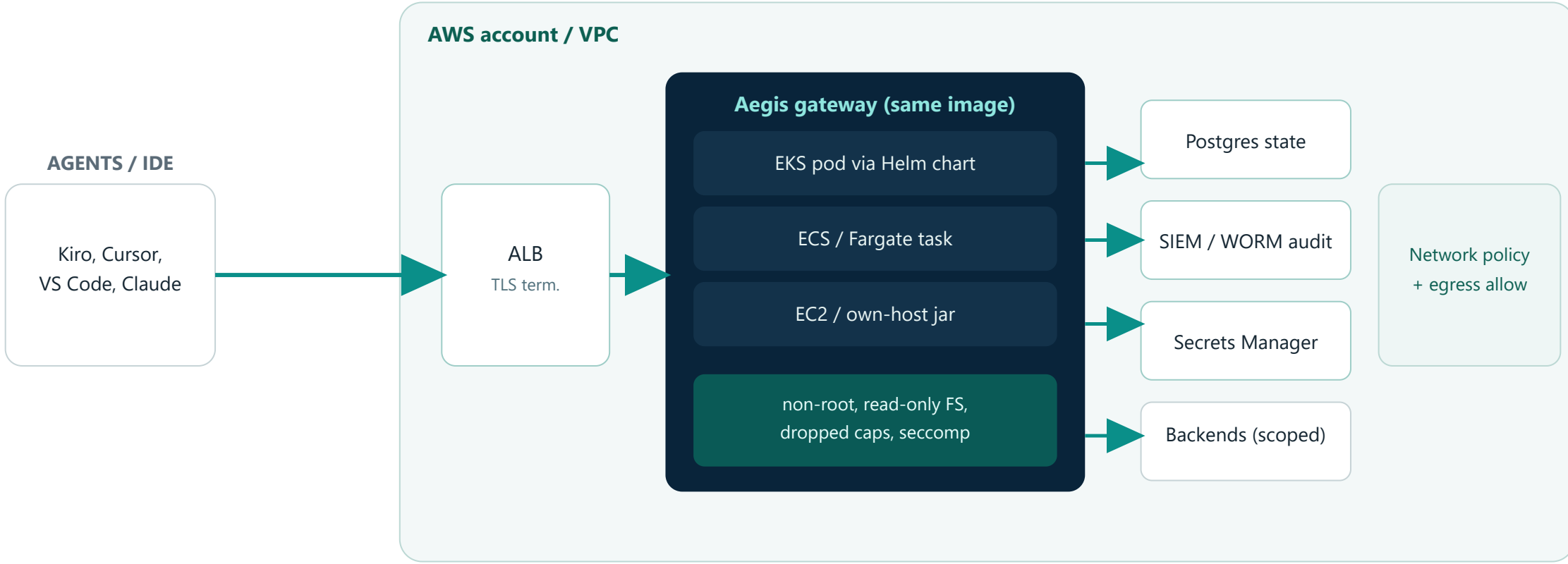
How a destructive call becomes a safe, approved action.



Fail-closed by design: any step can stop the call, and every decision is written to the audit journal.
The agent gets a clear, machine-readable reason, so a safe default never looks like an unexplained failure.

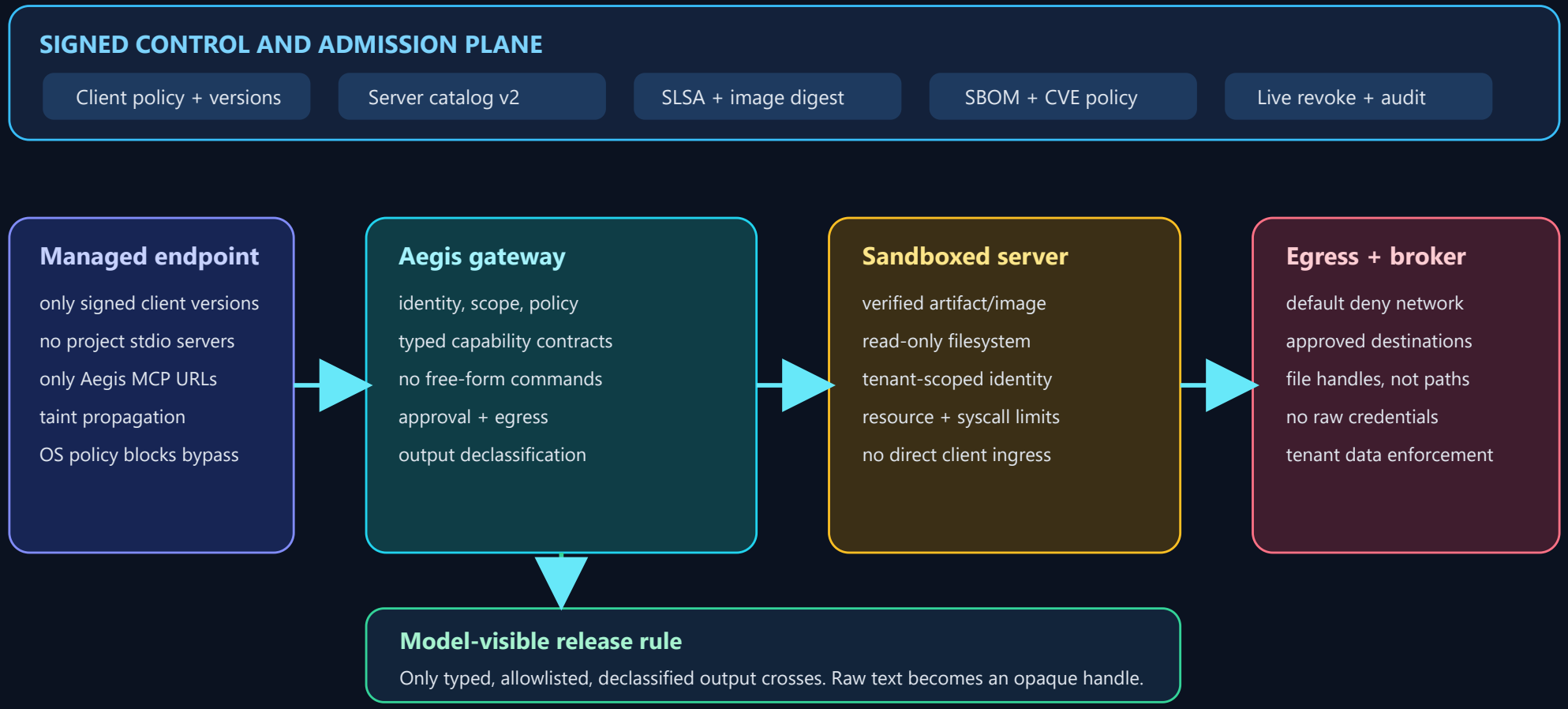
WHERE IT RUNS

One image: EKS, ECS or EC2, or own host.



COMPLETE CONTROL

The full boundary: endpoint to egress.



FLEXIBLE FRONT DOOR

Own host or Apache Knox, same engine.

Own-host HTTP

A standalone server with TLS, JWT/JWKS or mTLS/SPIFFE identity, route groups, CORS and an optional separate admin listener. Ideal for a quick start and container deployments.

Apache Knox

Drop the same governance engine inside an existing Knox topology for perimeter, identity and route admission, with Knox-path token revocation. No Knox classes in the own-host jar.

Both doors feed one fixed governance core, so a profile you prove on your laptop moves to either front door without change.

THE GOVERNANCE MODEL

Identity to audit, every call.

- **Identity required.** JWT/JWKS, mTLS/SPIFFE, static or Knox subject.
- **No token passthrough.** Backends get a scoped, gateway-issued credential.
- **Default deny.** Only configured tools, classes, resources and destinations are reachable.
- **Separation of duties.** Destructive calls need an independent approver and a receipt.
- **Catalog pinning.** A changed tool is quarantined; a signed catalog decides what starts.
- **Egress control.** URLs and DNS classified, redirects not followed, DNS pinned.
- **Output declassification.** Redaction, structured allowlist, or handle-only.
- **Properties S1 to S31.** Each names an enforcing class and a test that stays green.

INSIDE THE BUILD

Eight core modules, one engine decides.

Module	Role	Enforces
aegis-engine	The decision core	Policy, scope, approval, catalog integrity, egress, output policy, audit
aegis-mcp	MCP protocol and clients	Transport, HTTP/stdio sources, pinned HTTP, process launcher
aegis-config	Configuration and wiring	Sources, hooks, secrets, token exchange, artifact admission
aegis-mcp-security	Perimeter and identity	JWT/JWKS, pre-auth limits and lockout, scope enforcement
aegis-gateway	Own-host server + CLI	Route groups, admin listener, CSRF/Origin, startup checks
aegis-knox3-extension	Apache Knox front door	Knox identity bridge, route admission, revocation
aegis-source-a2a	Agent-to-agent source	A2A agents as governed tools
aegis-observability-otlp	Telemetry	OTLP span export of decisions

Optional modules add Derby and Postgres state, JDBC and ZooKeeper sources, and guardrails.

A comprehensive control set, added and tested.

Network and identity
DNS and egress pinning, JWT jti with revocation across the delegation chain, per-address and IPv6 /64 pre-auth limits, pinned JWKS, authenticated egress proxy.

Admission and output
Signed deployment attestation, host-anchored plugin digest, execution contracts, structured-allowlist and handle-only output, AES-256-GCM for retained handles.

Evidence and audit
Signed audit checkpoints, a SIEM sink interface, and the complete-control profile that enforces all of the above together.

Seven passes of hardening, each validated with a green build, bring these controls together under one profile.

ENGINEERING QUALITY

Every build passes four gates.

SpotBugs + FindSecBugs
Static and security analysis at maximum effort; real defects fixed, exclusions justified.

JaCoCo
Coverage enforced as part of the build.

Apache RAT
License headers checked on every source file.

CodeQL + OSV
Deep security and dependency scanning as CI workflows.

Clean result
The first SpotBugs run reported 72 findings. The genuine defects (constant-time digest comparison, a regex performance path, null guards, dead code) were fixed, and every remaining exclusion carries a written reason.

Green build
408 tests, 0 failures, across the eight core modules on 2026-11-07, with all four gates on in one run.

STANDARDS COVERAGE

Strong, tested OWASP coverage.

Full gateway path

MCP01 tokens, MCP02 privilege, MCP07 authn/authz, MCP08 audit, LLM05 output handling, LLM06 excessive agency, LLM10 unbounded consumption.

Full managed path

MCP03 tool poisoning, MCP04 supply chain, MCP05 command injection, MCP06 prompt injection, MCP10 over-sharing, LLM02 disclosure.

Complementary controls

Seven risks (shadow servers, direct injection, model supply chain, poisoning, system prompt, vector stores, misinformation) are owned with your endpoint, network and model providers. Aegis contributes to each and names the owner.

Headline

13 of 20 risks closed on the gateway path, with the gateway-ownable part of every other risk implemented and tested.

THREAT MODEL

STRIDE, named abuse cases, a precise claim.

STRIDE per zone becomes abuse cases TM01 to TM10, each with preconditions, attacker goal, expected outcome and the owning property.

- TM01 prompt-injected destructive call
- TM02 tool poisoning and rug pull
- TM03 confused deputy, TM04 SSRF
- TM06 self-approval, TM07 shadow MCP

The claim, under complete control

No unadmitted tool request and no raw, untrusted tool result crosses the Aegis-controlled boundary. Isolation and least privilege keep any defect in admitted software inside its assigned capability.

A confident, bounded promise: the managed path is controlled end to end.

PROOF, NOT CLAIM

Real exploits, stopped in a live run.

CVE class	Direct (no gateway)	Through Aegis
Filesystem path traversal (CVE-2025-53109/53110)	reads a secret file	DENIED SCOPE_DENIED
Command injection (CVE-2025-69256)	shell chain executes	DENIED READONLY_CALLER / APPROVAL_REQUIRED
Arbitrary SQL / git server (CVE-2025-68143/68145)	`DROP TABLE` executes	DENIED INVALID_INPUT
SSRF to cloud metadata	request is made	DENIED EGRESS_DENIED
Secret disclosure / tool poisoning	key, PEM, hidden instruction returned	CONTAINED REDACTED / CATALOG_INTEGRITY / HANDLE_ONLY

Verbatim from a real run of the 2.0 build. Reproducible with one command. Aegis complements client-side protections for client-only CVEs.

COVERAGE AT SCALE

Mapped against the real MCP CVE landscape.

We built a dated census of MCP-related CVEs from the national vulnerability database and mapped each one to how Aegis handles it. The result shows the breadth of the path Aegis governs.

561
unique MCP-related CVEs catalogued

22
closed on the gateway path today

297
closed on the managed (complete-control) path

319
fall on a path Aegis governs

Every row records product, versions, CWE, CVSS, attack boundary and the Aegis control. Dated 2026-11-07.

RESPONSIBLE AI

Governance where the agent acts.

Human oversight
Independent approval and dry-run receipts for consequential actions.

Accountability
Hash-chained audit, provenance on results, explainable decisions.

Privacy
Secret and PII redaction, handle-only output, least privilege.

NIST AI RMF	Aegis contribution
Govern	Policy as configuration, separation of duties, fail-closed defaults
Map	Typed catalog of exactly which tools, resources and destinations are reachable
Measure	Audit journal, decision records, control suite and CVE containment run
Manage	Approval, revocation, rate limits, break-glass containment

An enabling layer for EU AI Act human-oversight and record-keeping obligations.

EVIDENCE

Proven, on the record.

408
tests, 0 failures, 8 core modules

4
build gates, all green

13/20
OWASP MCP+LLM risks closed

0
attacks reached the backend through Aegis

Recorded and reproducible

The hardened 8-module build, the S1 to S31 control suite, and the before-and-after CVE containment run on the real jar, each with retained evidence.

Transparent

Every number ties to a dated log or test. Where a check needs a cluster or a CI service, that is stated plainly, so the evidence is trustworthy.

ADOPTION

Connect an AI IDE in minutes.

To the IDE, Aegis is a normal MCP server over HTTP. Point the client at the gateway URL and present a bearer token. Aegis decides the rest.

- Kiro and Cursor: `url` plus `Authorization: Bearer`.
- VS Code: a prompted secret input plus an `http` server entry.
- Claude Desktop and the MCP Python SDK: the same URL and token.
- Ready-made settings ship for 27 Apache projects, own host and Knox.

The token is the identity

Its groups decide which tool classes it may use, whether a write needs an approver, and what it may read. Issue a narrow token per person or agent.

Deploy on AWS, then swap the local URL for the load balancer address. Nothing else changes.

ECOSYSTEM

Proven across the Apache data stack.

Aegis ships one IDE-ready tutorial per verified project, each built from a retained real run through both front doors, with the engine command line, prompts mapped to tools, and the full request table.

Streaming
Kafka, Pulsar, Flink, Storm, NiFi, Flume

Lakehouse
Spark, Iceberg, Hive, Doris, Druid, Pinot

Data and search
HBase, Hadoop, CouchDB, Solr, Ignite, Calcite

Platform
ZooKeeper, BookKeeper, Airflow, Ranger, Superset, ActiveMQ

27 project tutorials, each with VS Code, Kiro and Cursor settings, plus an existing-MCP-server walkthrough.

CLEAN FIT

Aegis plus your existing controls.

Aegis is strongest as the governance layer in a defense-in-depth stack. It owns the MCP action boundary and integrates cleanly with the controls you already run.

You already have	Aegis adds
WAF and API gateway	Semantic control inside the MCP JSON-RPC body that a transport gateway cannot read
Identity provider	Per-call policy, scope and approval, with no token passthrough to backends
SIEM	A hash-chained decision journal and a sink that feeds your SIEM directly
Network and endpoint policy	The in-path controls that make those policies effective for agent traffic

ROADMAP

Strong today, and getting stronger.

Stage	State	What it adds
G1 Semantic gateway	Shipped	Identity, policy, scope, approval, egress, catalog pins, audit, output control
G2 Admitted and isolated servers	In progress	Artifact admission, process isolation, backend network policy, scoped credentials
G3 Managed endpoint	Designed	Endpoint guard, client admission, taint propagation, strict release rule
CI and supply chain	Workflows added	CodeQL, OSV scan, Helm render gate

Every stage is useful on its own, and each one builds toward complete control of the managed path.

GETTING STARTED

Four steps to a governed agent.

1 Start
Run the gateway image locally or on AWS with one configuration file.

2 Connect
Point your IDE agent at the gateway URL with a bearer token.

3 Govern
Declare tools, classes, scopes and approvers; Aegis enforces them.

4 Prove
Review the audit journal and the decision receipts for every action.

A profile you prove on a laptop moves to EKS, ECS or EC2 unchanged. Start small, scale with confidence.

SUMMARY

One governed route. Proven, and ready.

Aegis turns an agent's power over real systems into controlled, approved, audited actions. The same image runs on EKS, ECS, EC2 or your own host, in front of an existing MCP server or an Apache project, with no change to the governed code.

408 tests on 8 modules with four gates green, 13 of 20 OWASP risks closed on the gateway path, 561 MCP CVEs mapped, and a live run that denied three real MCP CVE classes.

Deploy on AWS | Connect your IDE | Run the CVE demo | Read the threat model